Retrieval Strategy, Not Just Retrieval: A Controlled Comparison of RAG Pipelines for Detecting Malicious PyPI Packages

Anonymous Author(s)
Submission Id: EASE2025-XXX

Abstract

Large language models (LLMs) can inspect source code for malicious behavior, but they miss subtle attacks such as obfuscated droppers and typosquats whose installation scripts look innocuous. Retrieval-augmented generation (RAG) has been proposed as the remedy, yet prior work couples retrieval to particular knowledge sources and mechanisms, leaving it unclear how much of the benefit comes from retrieval itself and how much from how retrieval is done. We isolate retrieval strategy through a controlled comparison of seven pipelines—a zero-shot baseline and six RAG variants (dense, sparse BM25, hybrid fusion, behavior-summary, contrastive dual-pool, and their combination)—with a fixed generator (Llama-3.1-8B-Instruct), a fixed code-only knowledge base of 4,163 training packages, and fixed prompting, evaluated on 1,020 test packages from the EASE 2025 dataset. Findings: any code-level retrieval raises malicious-class F1 from 0.888 to 0.968–0.972, a recall gain of 17 points; sparse BM25 matches a 4B embedding retriever within 0.4 F1 points, and hybrid fusion adds nothing; contrastive retrieval lowers the false-positive rate to 0.1% (0.948 recall, 0.996 precision); and converting queries to AST behavior summaries is actively harmful (F1 0.878). We further analyze the samples that no method detects (metadata-cloned typosquats) and use a static-grounding proxy to measure explanation faithfulness, showing that retrieval also reduces unsupported behavior claims. Our replication package publishes per-sample outputs and evaluation scripts.

CCS Concepts

• **Software and its engineering** → **Software development techniques**; • **Security and privacy** → *Systems security*.

Keywords

supply chain, malicious package, PyPI, large language model, RAG, retrieval, typosquatting

ACM Reference Format:
Anonymous Author(s). 2025. Retrieval Strategy, Not Just Retrieval: A Controlled Comparison of RAG Pipelines for Detecting Malicious PyPI Packages. In *Proceedings of Evaluation and Assessment in Software Engineering (EASE 2025).* ACM, New York, NY, USA, 6 pages. https://doi.org/XXXXXXX.XXXXXXX

EASE 2025, TBD

ACM ISBN 978-1-4503-XXXX-X/2025/06
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Open-source package registries such as PyPI are a primary vector for software supply-chain attacks: adversaries publish packages that masquerade as popular libraries—through typosquatting, name squatting, or dependency confusion—and execute malicious code during installation. Studies of real-world registries have documented thousands of such incidents [6, 9], and the attack surface keeps growing because anyone can publish, and installation scripts (`setup.py`) are arbitrary code. Manual review does not scale, which has motivated static-analysis heuristics and, more recently, large language models (LLMs) that classify source code as malicious or benign.

LLMs are promising but imperfect reviewers: an 8B-scale instruct model is strong at recognizing overt malware, yet it misses subtle cases such as obfuscated droppers or typosquats whose installation script looks, on its face, like any other package [1]. One prominent remedy is retrieval-augmented generation (RAG): before deciding, show the model similar examples retrieved from a corpus of previously seen malware [1, 13]. Recent work applying RAG to malicious PyPI package detection, most directly the EASE 2025 study "Detecting Malicious Source Code in PyPI Packages with LLMs: Does RAG Come in Handy?" [1], reports that retrieved context improves detection, but that study coupled retrieval to particular knowledge sources (external threat-intelligence collections such as YARA rules and GitHub advisories) and a single retrieval mechanism, leaving an important question open: *how much of the benefit comes from retrieval itself, and how much from how retrieval is done?*

> *Research question.* Given a fixed code-only knowledge base and a fixed LLM, which retrieval strategy—none, dense, sparse, hybrid, behavior-based, or contrastive—best detects malicious PyPI packages, and how do the strategies trade off recall, false alarms, and explanation faithfulness?

Figure 1: The research question addressed in this paper.

This paper reports a controlled comparison of seven pipelines that differ only in their retrieval strategy, evaluated on the same dataset used by the EASE 2025 study, with a single fixed LLM (`Llama-3.1-8B-Instruct`) and a knowledge base built purely from the dataset's own training partition. Our results show that:

- **Any code-level retrieval is a large win.** Dense or sparse retrieval lifts malicious-class F1 from 0.888 (no RAG) to 0.968–0.972 (+8 points), driven almost entirely by a 17-point recall gain (0.798 → 0.972).
- **Sparse retrieval suffices.** Plain BM25 over source tokens matches a 4B embedding model within 0.4 F1 points, and hybrid fusion of the two does not improve on the better single retriever.

- **Contrastive retrieval minimizes false alarms.** Retrieving benign counterexamples alongside malicious ones lowers the false-positive rate from 0.8–1.2% to 0.1% while preserving 0.948 recall and 0.996 precision.
- **AST behavior summaries are lossy.** Representing code as extracted behavior summaries for retrieval *hurts* (F1 0.878), below even the no-RAG baseline.

2 Motivation

Two practical concerns motivate the comparison. First, deployment cost: if a lexicon index suffices, teams can skip embedding infrastructure, vector stores, and the associated hosting budget. Second, triage cost: security-reviewing LLM-predicted alarms is expensive, so the false-positive rate per millisecond-of-recall may matter more than raw F1. By measuring recall, false alarms, coverage (parse failures), and explanation grounding side by side, we aim to give practitioners—and follow-up researchers—a basis to choose a retrieval strategy on purpose rather than by habit.

3 Contributions

The main contributions of this paper are:

- A controlled, reproducible comparison of six RAG variants plus a zero-shot baseline (E0–E6) for LLM-based detection of malicious PyPI packages on the EASE 2025 dataset, isolating retrieval strategy as the only varying factor.
- Evidence that sparse retrieval matches dense retrieval on this task (0.968 vs. 0.972 F1) with no learned components.
- Evidence that contrastive dual-pool retrieval reduces false alarms by an order of magnitude at modest recall cost.
- A negative result for behavior-summary retrieval, quantifying what is lost when code is reduced to AST-derived descriptions.
- An analysis of replication-grade failure modes: samples no method detects (metadata-cloned typosquats), recurring false positives, and structured-output parse failures, with per-sample raw outputs and a reproducible evaluation pipeline released as a replication package.

4 Paper Structure

The remainder of this paper is structured as follows. Section 5 reviews related work on supply-chain attacks, LLMs for malicious-code detection, and retrieval-augmented methods. Section 6 details the dataset, the seven retrieval strategies, the LLM configuration, and the evaluation metrics. Section 7 presents the main results, fairness-oriented faithfulness statistics, and an error analysis. Section 8 interprets the findings, discusses limitations and threats to validity, and outlines future work. Section 9 concludes.

5 Related Work

5.1 Supply-Chain Attacks on Package Registries

Malicious publications in package registries are a well-documented, persistent threat. Duan et al. [6] measured hundreds of malicious packages across PyPI, npm, and RubyGems, showing that most are installed by ordinary users through ordinary `pip install`-style commands, and that name-based imitation (typosquatting) is a dominant attack style. Ohm et al. [9] cataloged attack techniques in open-source software supply chains, including injections that trigger at package-install time through `setup.py` hooks. Industry efforts such as Datadog's malicious-software-packages dataset [5] have made labeled collections of such packages publicly available, which we use here.

5.2 LLMs for Malicious-Code Detection

Recent work applies LLMs to security classification tasks. Studies on vulnerability reasoning caution that LLMs remain unreliable on subtle security judgments without auxiliary signal [12]. For malicious packages specifically, the EASE 2025 study—the closest prior work to ours—treated package classification as a zero-shot and retrieval-augmented task, evaluating open and proprietary models on a dataset of malicious and benign PyPI packages [1]. That study augmented generation with knowledge bases built from YARA rules, GitHub security advisories, and malicious `setup.py` examples, and reported both that LLM-only detection leaves room for improvement and that retrieved context helps. A complementary line of work applies multi-agent LLM pipelines to package-level evidence such as metadata and behavioral descriptions, reporting improvements from aggregating multiple signals (`LAMPS`, "Many Hands Make Light Work") [2]. Our work differs in scope and design: instead of changing the knowledge source or inventing new pipelines, we fix the corpus, the model, and the prompt, and vary only the retrieval strategy, which lets us attribute performance differences to retrieval design rather than to data or agent orchestration.

5.3 Retrieval and Retrieval-Augmented Generation

Retrieval-augmented generation combines a retriever with a generator so that the model conditions on relevant external evidence. Dense passage retrieval established embedding-based retrieval as a standard for open-domain question answering [7]; in parallel, classical sparse scoring, notably the Okapi BM25 function, remains a strong lexical baseline with a well-understood probabilistic foundation [10]. Hybrid systems fuse the two, most simply with Reciprocal Rank Fusion [4]. In the RAG literature for security-adjacent code tasks, retrieving code examples has repeatedly been the first thing tried [1, 13], yet—to our knowledge—no prior study has systematically compared dense, sparse, hybrid, behavior-based, and contrastive retrieval configurations on the same malicious-package benchmark with the same generator. This is the gap we address.

5.4 Summary

In summary, prior work (i) established the supply-chain threat and the datasets we build on, (ii) showed that LLMs augmented with retrieved knowledge are promising but not sufficient for malicious-package detection, and (iii) provides the retrieval toolbox (dense, sparse, fusion). Our contribution is a controlled, seven-condition comparison that isolates retrieval strategy, together with faithfulness-oriented evaluation of the model's behavior claims, on the same data and model family used in the EASE 2025 study.

6 Methodology

6.1 Overview

We study whether, and how, retrieval-augmented generation (RAG) improves an LLM's ability to detect malicious PyPI packages, by comparing seven alternative pipelines E0–E6 that differ only in the *retrieval strategy* they apply before the final classification. At a high level, every pipeline (1) takes the target package's `setup.py` source as input, (2) builds a query representation from it, (3) optionally retrieves reference examples from a knowledge base of previously seen packages, (4) feeds the target together with the retrieved evidence (if any) to an instruction-tuned LLM, and (5) asks the model for a structured verdict: malicious or benign, a confidence score, security-relevant behaviors with source-line citations, and a rationale. Figure 2 illustrates the pipeline.

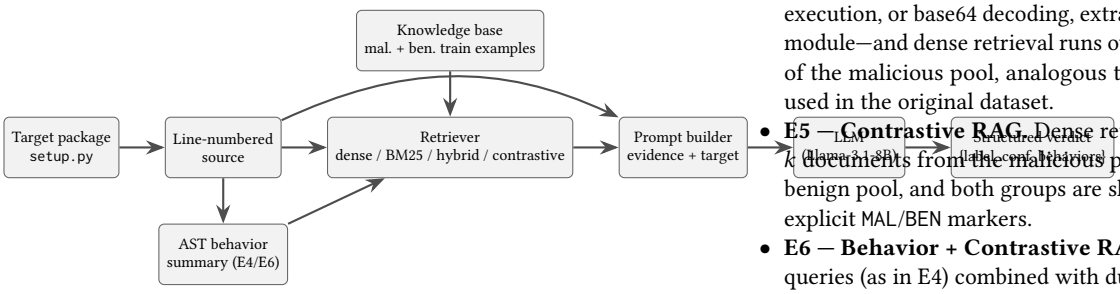

Figure 2: Overview of the evaluated pipelines. All stages are shared across methods except query construction and retrieval (E4/E6 additionally build an AST behavior summary as the query).

6.2 Dataset

We reuse the dataset assembled in the EASE 2025 study on RAG for malicious code detection [1], which itself mines packages from Datadog's malicious-software-packages dataset [5]. The replication package ships pre-split JSON files: a training partition of 1,158 malicious and 3,005 benign packages, and a test partition of 276 malicious and 753 benign packages (1,029 in total). Each entry contains the package name, file list, and the raw `setup.py` content; test entries additionally carry an AST-derived textual behavior description. Nine malicious test entries whose `setup.py` could not be extracted (content string `"Not Available"`) are excluded from evaluation, leaving 1,020 test samples. The training partition serves as the knowledge base; the test partition is never used for retrieval.

The malicious class is dominated by typosquatted or squatted packages (e.g., `colorama` clones, `requests`-lookalikes) whose installation code frequently hides its malicious behavior in the `setup.py` entry point. This makes the task representative of real-world supply-chain attacks that LLM-based tooling is expected to catch.

6.3 Retrieval Strategies

All methods share the same knowledge base: the raw `setup.py` sources of the 4,163 training packages, indexed in separate malicious and benign pools. We fix the number of retrieved documents to $k = 3$, following the reference design. The seven methods are:

- **E0 — No RAG.** The target source is classified directly, without any retrieved context. This is the controlled baseline.
- **E1 — Dense RAG.** The query is embedded with a sentence embedding model and matched by cosine similarity against the embedded training documents. We use the `Qwen3-Embedding-4B` model [11] and a FAISS `IndexFlatIP` store over L2-normalized vectors [8].
- **E2 — BM25 RAG.** Sparse lexical retrieval with the Okapi BM25 scoring function [10]; no learned components.
- **E3 — Hybrid RAG.** Dense (E1) and sparse (E2) rankings are fused with Reciprocal Rank Fusion [4], $\sum_r 1/(60 + \text{rank}_r(d))$; no fusion weights are trained.
- **E4 — Behavior RAG.** The target is represented by a static behavior summary—imports, suspicious API calls, and behaviors such as shell execution, network access, dynamic execution, or base64 decoding, extracted with Python's `ast` module—and dense retrieval runs over behavior summaries of the malicious pool, analogous to the AST descriptions used in the original dataset.
- **E5 — Contrastive RAG.** Dense retrieval is run twice: top-k documents from the malicious pool *and* top-k from the benign pool, and both groups are shown to the model with explicit `MAL`/`BEN` markers.
- **E6 — Behavior + Contrastive RAG.** Behavior-summary queries (as in E4) combined with dual-pool contrastive evidence (as in E5).

6.4 LLM Configuration and Prompting

Classification is performed by `Llama-3.1-8B-Instruct` [3] served through an OpenAI-compatible vLLM endpoint [8]. Generation uses temperature 0 and a maximum of 1,024 output tokens. The system prompt instructs the model that it is performing static source-code analysis, not to assume behavior unsupported by the source or retrieved evidence, and that retrieved examples are references rather than proof; every claimed behavior must cite target source lines. The target source is line-numbered before it is embedded in the prompt. The output schema is enforced as JSON and requires `verdict` (malicious/benign), `confidence`, `behaviors` (a list of {type, evidence} objects) and `rationale`. Retrieved documents are labeled with stable identifiers (`MAL_*` / `BEN_*`) and their retrieval scores.

6.5 Evaluation Metrics

Classification. We report precision, recall, and F1 for the malicious class, benign-class F1, macro-F1, balanced accuracy, false-positive rate (FPR), and false-negative rate (FNR). Because JSON decoding can fail (e.g., output truncation by the serving backend), we additionally report *coverage*: the fraction of samples whose response parsed successfully; unparsed samples are counted as failures and excluded from metric denominators, so that success and failure modes are reported separately.

Explanation faithfulness. As an approximation of whether the model "hallucinates" security-relevant behaviors, we ground each claimed behavior in a lightweight static checker built on Python's `ast` module, which flags shell execution, process creation, network access, file writes/deletion, dynamic execution, environment access,

base64 decoding, remote download, and persistence. Free-form behavior names produced by the model are keyword-normalized to this vocabulary; a claim is *supported* when the corresponding static flag is present on any line of the target. We report the unsupported-claim rate = unsupported/claimed, behavior precision and behavior recall. Because static analysis is neither sound nor complete on obfuscated code, we treat these figures as an *automated static-grounding proxy*, not as hallucination ground truth.

6.6 Implementation and Reproducibility

The pipeline is implemented in Python 3.12 as a modular package (retrieval, static analysis, prompting, evaluation modules) without framework dependencies; BM25 uses `rank-bm25`, dense indexes use `faiss`, and LLM/embedding calls go through a single OpenAI-compatible client so that any endpoint can be swapped by environment variables. All LLM responses, embeddings, and retrieval results are cached (SQLite) keyed by prompt hash, sample content, evidence IDs, and generation parameters; retrieval indexes are persisted on disk; and every run records its configuration, seed, model IDs, and git commit. Run scripts for all seven methods, the evaluation code, and raw per-sample outputs are included in the replication package.

7 Results

7.1 Experimental Setup

We evaluate seven methods (E0–E6, Section 6.3) on the 1,020 usable test samples (267 malicious, 753 benign), all served by the same `Llama-3.1-8B-Instruct` model with identical prompting, decoding parameters ($T = 0$, 1,024 max tokens), and $k = 3$ retrieval settings, so that the only varying factor is the retrieval strategy. Dense indexes use `Qwen3-Embedding-4B`; BM25 uses Okapi BM25 with a standard implementation. The nine samples with unavailable source are excluded from every method.

7.2 Main Results

Table 1 reports classification performance. Three findings stand out.

Table 1: Classification results on the EASE test set (1,020 samples; 267 malicious). Coverage is the fraction of samples whose structured response parsed successfully; metrics are computed over parsed samples. Best value per column in bold.

Method	Mal. P	Mal. R	F1	Ben. F1	BAcc	FPR	Cov.
E0 No RAG	**1.000**	0.798	0.888	0.973	0.899	**0.0%**	0.933
E1 Dense	0.972	**0.972**	**0.972**	**0.991**	**0.981**	0.9%	0.975
E2 BM25	0.964	**0.972**	0.968	0.989	0.980	1.2%	0.975
E3 Hybrid	0.976	0.964	0.970	0.990	0.978	0.8%	0.976
E4 Behavior	**1.000**	0.782	0.878	0.964	0.891	**0.0%**	**0.977**
E5 Contrastive	0.996	0.948	0.971	0.990	0.973	**0.1%**	0.927
E6 Beh.+Contr.	**1.000**	0.788	0.881	0.966	0.894	**0.0%**	0.974

Finding 1: retrieval helps, and code-level retrieval helps a lot. The zero-shot baseline (E0) achieves F1 0.888 with perfect precision but misses roughly one in five malicious samples (recall 0.798). Dense retrieval (E1) lifts malicious recall to 0.972 and F1 to 0.972 (+8.4 points) while benchmarking near-perfect balanced accuracy (0.981); BM25 (E2) and hybrid (E3) essentially match this level (F1 0.968–0.970). This confirms that retrieval is useful for malicious-code detection, and shows that the gain comes almost entirely from exposing the model to similar *code* rather than from any particular similarity mechanism.

Finding 2: sparse retrieval is not worse than dense retrieval. BM25 (0.968) is within 0.4 F1 points of dense (0.972) at a fraction of the engineering cost—an in-memory term index, no embedding model, no vector store. Fusing both (E3) does not beat the better of the two, consistent with prior RAG literature on short, keyword-heavy security queries. For this task, lexical similarity over code tokens is a strong and cheap baseline that future work should not skip.

Finding 3: contrastive evidence trades recall for the fewest false alarms. E5, which shows the model both malicious and benign reference groups, yields the best precision among RAG methods (0.996) with FPR of 0.1%—an order of magnitude below dense/BM25/hybrid (0.8–1.2%)—at the cost of recalling slightly fewer malicious samples (0.948 vs. 0.972). In a setting where triaging false alarms dominates cost, E5 dominates.

Finding 4: static behavior summaries are lossy. Replacing the query with an AST-derived behavior summary (E4) or adding it on top of contrastive retrieval (E6) actively hurts: F1 drops to 0.878/0.881, below the no-RAG baseline in recall-adjusted terms, although all misclassifications remain false negatives and FPR stays at zero. The summaries discard identifier names, call structure and file-path information that the model uses to recognize attacks such as typosquats; retrieving behavior-summary neighbors does not compensate.

7.3 Faithfulness of Explanations

Table 2 reports the static-grounding proxy computed by keyword-normalizing the model's free-form behavior claims to our static vocabulary and checking each against `ast`-derived behavior flags. Unsupported-claim rates are high across the board (0.59–0.72), a caveat we discuss in Section 8, but the ranking is informative: the no-RAG baseline is the least grounded (0.717), all retrieval variants reduce unsupported claims, and behavior-based queries (E4) yield the lowest rate (0.593) with the best claim precision pair (E4/E5: 0.387/0.421). Retrieving explicit evidence therefore correlates with more grounded, less extrapolated explanations, and behavior prompts in particular restrain the model to security-relevant vocabulary.

7.4 Error Analysis

Consistently missed samples. Five malicious packages are misclassified as benign by *all seven* methods: `parseweb-v3`, `kazer12-v2`, and `testdufou-v2` are near-verbatim metadata clones of the legitimate `colorama` package (same authors, description and project URL in `setup.py`) that differ only by name; `pepequests-v0.0.1` is

Table 2: Automated static-grounding statistics for model-claimed behaviors (security-relevant, normalized claims only). `Unsup.` = unsupported claimed behaviors / all claimed; `Beh. P` = supported claims / claimed; `Beh. R` = claimed-and-supported behaviors / statically observed behaviors.

Method	Claims	Unsup.	Beh. P	Beh. R
E0 No RAG	427	0.717	0.282	0.427
E1 Dense	560	0.657	0.312	0.521
E2 BM25	596	0.685	0.294	0.517
E3 Hybrid	568	0.665	0.314	0.525
E4 Behavior	526	**0.593**	0.387	**0.531**
E5 Contrastive	455	0.615	**0.421**	0.457
E6 Beh.+Contr.	697	0.694	0.289	**0.543**

a minimal typosquat of `requests` whose `setup.py` contains no executable behavior at all; and `ucap-v3.6.1` hides its malicious setup behind innocuous packaging boilerplate. One further typosquat, `simplyjson`, is missed by six of seven methods. These failures are structural: when the *specific file we analyze* carries no distinguishing signal, no retrieval over the same file type can help, and name-level analysis (e.g., similarity to popular packages) would be required.

Recurring false positives. Two benign packages are flagged as malicious by three of seven methods (`authcaptureproxy-1.1.4`, `undetected-chromedriver-3.1.5.post4`), both of which legitimately exercise capture-proxy and privilege-heavy browser automation—code patterns orthographically similar to credential-theft trojans in the knowledge base. Spot checks of the sampled false positives suggest they arise from the model over-extending retrieved malicious evidence, the failure mode that contrastive retrieval (E5) is designed to suppress: indeed E5 flags neither of these two packages.

Parse failures. Structured-output decoding fails on 2.3–7.3% of samples depending on the method. The no-RAG baseline (6.5%) and contrastive method (7.3%) are worst; both correspond to the largest average prompt complexity (long obfuscated targets for E0; dual evidence blocks for E5), consistent with backend truncation of long outputs. We treat coverage as a first-class result rather than silently dropping failed samples.

8 Discussion

8.1 Interpretation of Results

Retrieval presence matters more than retrieval mechanism. Across dense, sparse, and hybrid variants, malicious recall settles around 0.96–0.97 versus 0.80 with no retrieval—the decisive factor is that the model sees similar code at all, not which similarity function found it. We attribute this to the structure of the problem: passages of malicious source are frequently near-duplicates (copy-pasted droppers, shared obfuscation snippets), which both lexical and semantic similarity recover equally well.

Sparse retrieval is the pragmatic default. BM25 has no embedding model, no GPU, and no index-approximation tuning, yet yields 0.968 F1 vs. 0.972 for a 4B embedding model. For researchers and practitioners who want a replication-friendly pipeline, we recommend starting with BM25 and upgrading only if a failure analysis shows semantically-paraphrased attacks slipping through.

Contrastive retrieval offers a tunable precision knob. Retrieving malicious examples *only* biases the model toward positives; adding retrieved benign look-alikes lets it weigh counterevidence. The 0.1% FPR of E5 is attractive when analysts must triage alarms, and the mechanism is orthogonal to retriever choice—a practitioner may obtain the same effect with BM25 over both pools.

Behavior summaries cripple the signal. The AST-based representations tested here (E4, E6) discard too much (identifiers, URL structure, file names, control flow). Their recall regresses to no-RAG levels while their recall-adjusted F1 falls below the baseline, which we interpret as evidence that queries should stay close to raw code whenever the generator can accept it.

8.2 Limitations

Our work has several limitations:

- **Single model and single dataset.** All findings are for one generator (`Llama-3.1-8B-Instruct`) and one benchmark; larger or stronger models may compress the gaps between retrieval strategies.
- **Single file granularity.** Analysis is limited to `setup.py`. Malware hidden in auxiliary files is invisible to every method tested, which partly explains the samples no method detects.
- **Fixed retrieval settings.** We fix $k = 3$ (per the reference design); a full $k \in \{1, 3, 5\}$ sweep and corpus-side ablations (e.g., leaving retrieved documents unlabeled) remain for future work.
- **AST-based query extraction is unrefined.** Our behavior summaries are simple import/call inventories; a richer, security-focused static analyzer might restore some of the lost signal.

8.3 Threats to Validity

Internal. Prompt wording is confounded with method (evidence blocks differ). We mitigated this through one shared system prompt, common output schema, stable document identifiers, and identical decoding parameters. Faithfulness scoring is an automated proxy: keyword normalization and AST detection are neither sound nor complete, so grounding numbers should be read comparatively, not absolutely. Parse failures were excluded from metric denominators and reported as coverage to avoid silently dropping or crediting failed samples.

Construct. "Ground-truth" labels originate from the reference dataset; samples whose code could not be extracted (9/1029) were removed, which slightly privileges clean verifiable cases. Classification error is kept distinct from explanatory hallucination: we score the verdict against the label and score the explanation separately against static evidence.

External. The malicious population is largely typosquats from a specific observation period; findings may not transfer to other attack families, registries, or later temporal windows. We therefore

treat the study as a controlled comparison rather than an absolute capability claim.

8.4 Future Work

Three directions follow directly. First, replicate over a model panel (2–4 generators) and a second dataset (e.g., LAMPS D1) to test whether strategy rankings generalize. Second, extend detection beyond `setup.py`: to whole-package multi-file evidence and, for the typosquat class that defeats every method here, to package-name similarity features. Third, bring the evidence pipeline closer to production: cross-dataset and temporal splits of the knowledge base, higher k with LLM-based relevance filtering cascades in the spirit of CRAG [13], and a deployed triage evaluation weighting FPR over retrieval efficiency.

8.5 Data and Code Availability

Raw per-sample outputs for all seven methods, the evaluation scripts, and run scripts are provided as a replication package alongside this report; the underlying dataset is the EASE 2025 replication package [1].

9 Conclusion

This paper asked what retrieval strategy, if any, best supports LLM-based detection of malicious PyPI packages. Using the EASE 2025 dataset, a fixed `Llama-3.1-8B-Instruct` generator, and a code-only knowledge base, we compared a zero-shot baseline against dense, sparse, hybrid, behavior-based, and contrastive retrieval.

Three conclusions appear robust. Any code-level retrieval is a large win, lifting malicious-class F1 from 0.888 to 0.968–0.972, mostly through recall. The mechanism matters surprisingly little: plain BM25 matches a 4B embedding retriever within 0.4 F1 points, so sparse retrieval is the pragmatic default. And contrastive retrieval is a precision dial worth turning: retrieving benign counterexamples cuts the false-positive rate to 0.1% at 0.948 recall. Conversely, reducing queries to AST behavior summaries is counterproductive (F1 0.878), demonstrating that signal loss in query construction, not retrieval per se, is where methods fail.

We also documented the residual failures that no retrieval over `setup.py` can fix—metadata-cloned typosquats—and the recurring false positives over security-adjacent legitimate packages, as guidance for the next generation of designs, which should extend evidence beyond a single file, add name-level signals, and calibrate retrieval toward triage workflows.

References

[1] [n. d.]. Detecting Malicious Source Code in PyPI Packages with LLMs: Does RAG Come in Handy?

[2] [n. d.]. Many Hands Make Light Work: An LLM-based Multi-Agent System for Detecting Malicious PyPI Packages.

[3] AI@Meta. 2024. The Llama 3 Herd of Models. https://arxiv.org/abs/2407.21783.

[4] Gordon V. Cormack, Charles L A Clarke, and Stefan Buettcher. 2009. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In *Proceedings of the 32nd International ACM SIGIR Conference*. 758–759.

[5] Datadog Security Labs. 2023. malicious-software-packages-dataset. https://github.com/datadog/malicious-software-packages-dataset.

[6] Ruian Duan, Omar Alrawi, Ranjita Pai Kasturi, Ryan Elder, Brendan Saltaformaggio, and Wenke Lee. 2021. Towards Measuring Supply Chain Attacks on Package Managers for Interpreted Languages. In *Network and Distributed Systems Security (NDSS) Symposium*.

[7] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 6769–6781.

[8] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*.

[9] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. 2020. Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks. In *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*. Springer, 23–43.

[10] Stephen Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval* 3, 4 (2009), 333–389.

[11] Qwen Team. 2025. Qwen3 Embedding Series. https://arxiv.org/abs/2505.13878.

[12] Saad Ullah, Mingji Han, Saurabh Pujar, Hammond Pearce, Ayse Coskun, and Gianluca Stringhini. 2024. LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities (Yet?): A Comprehensive Evaluation, Framework, and Benchmarks. In *IEEE Symposium on Security and Privacy (S&P)*.

[13] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. 2024. Corrective Retrieval Augmented Generation. https://arxiv.org/abs/2401.15884. In *Findings of the Association for Computational Linguistics: NAACL*.